

MVC Project: Multilayer Perceptron

Muhammad Hasnain Farooq

Roll No: 25i-0039 Section: AI-A Date: 29th March, 2026

Abstract. This report presents a comprehensive implementation and analysis of a Multilayer Perceptron (MLP) for both a small synthetic dataset and the MNIST handwritten digit dataset. The project begins with manual computation of forward propagation, loss evaluation using Mean Squared Error (MSE), and gradient computation through back-propagation. Optimization techniques including stochastic gradient descent, mini-batch gradient descent, Momentum, and Nesterov Accelerated Gradient are explored and compared.

The implementation is extended to the MNIST dataset using NumPy. The model achieves a final training loss of 0.0069 and a test accuracy of 95.54%, demonstrating effective learning and generalization. The results highlight the importance of activation functions, gradient-based optimization, and structured training procedures in neural network performance.

1 Introduction

Artificial Neural Networks are fundamental to modern machine learning systems. The Multilayer Perceptron is a widely used architecture for classification tasks.

This project aims to develop a deep understanding of neural networks by manually implementing forward propagation, loss computation, and backpropagation. These concepts are then extended to a large-scale dataset (MNIST) using Python and NumPy.

2 Background

2.1 Multilayer Perceptron

A Multilayer Perceptron consists of multiple layers of neurons. Each neuron computes a weighted sum followed by a nonlinear activation:

$$z = \sum w_i x_i + b \tag{1}$$

$$a = \sigma(z) \tag{2}$$

2.2 Activation Function

The sigmoid activation function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (4)$$

2.3 Loss Function

The Mean Squared Error is given by:

$$L = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2 \quad (5)$$

2.4 Backpropagation

Backpropagation computes gradients using the chain rule:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}} \quad (6)$$

$$\frac{\partial L}{\partial w} = \delta \cdot a_{\text{previous}} \quad (7)$$

2.5 Optimization Algorithms

Gradient Descent:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad (8)$$

Momentum:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t \quad (9)$$

$$w \leftarrow w - \eta v_t \quad (10)$$

Nesterov Accelerated Gradient:

$$\tilde{w} = w - \eta \beta v_{t-1} \quad (11)$$

$$g_t = \frac{\partial L(\tilde{w})}{\partial \tilde{w}} \quad (12)$$

$$v_t = \beta v_{t-1} + (1 - \beta) g_t \quad (13)$$

$$w \leftarrow w - \eta v_t \quad (14)$$

3 Implementation

3.1 Manual Forward Pass

The assigned parameters are:

$$\begin{aligned} w_1 &= 0.51, & w_2 &= -0.81, & w_3 &= 0.03, & w_4 &= 0.46 \\ w_5 &= 0.54, & w_6 &= -0.31, & w_7 &= 0.85, & w_8 &= 0.55 \\ w_9 &= 0.31, & w_{10} &= 0.73 \\ b^{(1)} &= -0.03, & b^{(2)} &= -0.15 \end{aligned}$$

Forward pass results:

Sample	x_1	x_2	$\hat{y}$
$s^{(1)}$	0.2	0.8	0.637
$s^{(2)}$	0.9	0.4	0.6298
$s^{(3)}$	0.1	0.2	0.6352

Table 1. Forward pass predictions

3.2 Loss Calculation

$$L = 0.2241 \tag{15}$$

3.3 Backpropagation and Training

Gradients were computed using the chain rule. The output layer error was propagated backward through hidden layers to compute weight updates.

Stochastic Gradient Descent showed consistent reduction in loss, while mini-batch training introduced small fluctuations due to averaging.

Optimizer comparison showed minimal difference due to dataset size.

3.4 Python Implementation

Network architecture:

- Input layer: 784
- Hidden layer 1: 128
- Hidden layer 2: 64
- Output layer: 10

Training setup:

- Learning rate: 0.1
- Epochs: 20
- Batch size: 32
- Weight initialization: Uniform $[-0.5, 0.5]$
- Bias initialization: zero
- Input normalization: $[0, 1]$

4 Experiments

4.1 Dataset

The MNIST dataset contains 60,000 training samples and 10,000 test samples. Each image is represented as a 784-dimensional vector.

4.2 Results

- Final training loss: 0.0069
- Test accuracy: 95.54%

4.3 Loss Curve

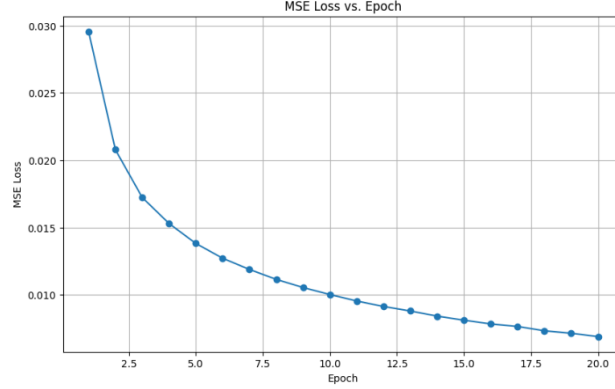


Fig. 1. Training loss vs epoch

The loss decreases smoothly across epochs, indicating stable convergence.

4.4 Sample Predictions

The model correctly classifies most digits, with occasional errors on visually ambiguous samples.

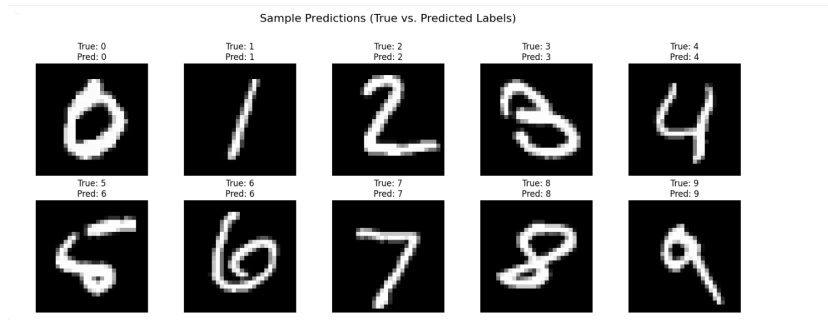


Fig. 2. Sample predictions on MNIST

5 Discussion

The manual implementation provides insight into how neural networks learn through gradient-based optimization. SGD demonstrates consistent improvement, while mini-batch training balances stability and efficiency.

The MNIST implementation shows strong performance despite a simple architecture. Minor misclassifications indicate potential improvements through deeper architectures or alternative activation functions.

6 Conclusion

This project demonstrates both theoretical and practical aspects of neural networks. The transition from manual computation to large-scale implementation validates the correctness of the approach.

The achieved accuracy confirms the effectiveness of the MLP model and training methodology.

References

1. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016)
2. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE (1998)